On-Device Characterization and Latency Profiling of Ultra-Low-Resource INT8 TinyML Models on ESP32 Microcontrollers

Narendra Satish

Abstract—Deploying deep learning on resource-constrained 32-bit microcontrollers (TinyML) requires empirical hardware characterization to bridge the substantial translation gap between host simulation and physical silicon. However, ultra-low-resource tabular diagnostic models (< 4 KB Flash, sub-100 µs execution latency) remain significantly under-characterized on commercial microcontrollers compared to vision and audio workloads. This paper presents a comprehensive on-device empirical characterization of four disk-verified FULL_INT8 Multi-Layer Perceptron (MLP) models deployed on an Espressif ESP32-D0WD-V3 microcontroller (dual-core Xtensa LX6 @ 240 MHz, 4 MB Flash, 320 KB SRAM). Using a high-precision zero-I/O in-RAM hardware timer benchmarking protocol across 24,000 measured single-sample inferences, we establish four primary findings: (1) isolated on-device latency scales strictly monotonically with parameter count ($R^2 = 0.963$) from 64.55 µs (176 parameters) to 89.90 µs (412 parameters); (2) structural knowledge distillation delivers an observed 28.20% physical execution latency reduction relative to the uncompressed baseline; (3) host x86_64 profiling underestimates physical microcontroller execution latency by 62.87 × –76.77× and introduces sub-microsecond rank inversions unmasked on silicon; and (4) TensorFlow Lite for Microcontrollers commits exactly 916 Bytes of static tensor arena memory with zero dynamic heap allocations across 25,200 consecutive executions. These empirical results provide rigorous, reproducible baselines for deploying real-time intelligence on edge cyber-physical and industrial diagnostic systems.

Index Terms—TinyML, Microcontroller Benchmarking, ESP32, Xtensa LX6, Integer Quantization, Latency Profiling, Embedded Intelligence, Edge Diagnostics.

I. Introduction

THE integration of Tiny Machine Learning (TinyML) to run trained models on low-power, resource-constrained 32-bit microcontrollers (MCUs) is emerging as a new paradigm in edge computing [1]–[3]. By executing inference on bare-metal silicon at the sensor edge, these edge devices provide extreme latency, zero telemetry bandwidth, reduced radio power, and privacy [4], [5]. For edge applications where the inference decisions are critical for safety or business operations such as high-frequency industrial vibration monitoring, engine diagnostics, acoustic anomaly detection, and embedded cyber-physical control, the edge devices must make inferences with tight latency budgets ($\leq$ 5–100 ms) and sub-milliwatt power consumption [9], [10].

Despite rapid algorithmic advances in neural network compression and pruning [20], a fundamental hardware translation gap persists between host-side software simulations and physical microcontroller execution [6], [9]. Host x86_64 development workstations employ deep out-of-order execution pipelines, multi-level branch predictors, multi-megabyte cache hierarchies, and advanced vector SIMD units (AVX2/AVX-512). These outboard microarchitectural features mask instruction-level bottlenecks, memory stall penalties, and register pressure that dominate embedded execution on deeply constrained 32-bit RISC microcontrollers [4], [7]. As a result, models that appear optimal during host-side training often demonstrate poor execution latency, memory overhead, or runtime nondeterminism when flashed to physical MCU hardware [14].

Furthermore, the existing TinyML benchmarking literature suffers from two notable limitations:

1) Workload Bias Toward Vision and Audio: Standard benchmark suites such as MLPerf Tiny focus on 2D Convolutional Neural Networks (CNNs) for visual wake words, image classification, and keyword spotting [4], [8]. These CNNs typically have $>$ 100 KB footprint in Flash ROM with inference latencies ranging from 10 ms to 300 ms. In comparison, ultra-low resource tabular diagnostic networks ($<$ 4 KB Flash, $<$ 100 µs execution latency) are largely uncharacterized on commercial silicon.
2) Methodological Timing Artifacts: Prior embedded benchmarking studies often log timestamps or intermediate results over serial UART buses as part of the measurement loop [3], [5]. UART transmission at typical baud rates (115,200 bps) introduces multi-millisecond driver blocking delays that overshadow true kernel execution time by orders of magnitude.

To address these challenges, this paper presents an empirical, publication-grade on-device characterization and latency profiling of four verified, disk-serialized FULL_INT8 Multi-Layer Perceptron (MLP) models deployed on physical Espressif ESP32-D0WD-V3 microcontroller silicon (dual-core Xtensa LX6 @ 240 MHz, 4 MB Flash, 320 KB SRAM). Figure 1 illustrates our physical deployment and zero-I/O measurement pipeline.

We structure our investigation around five concrete

Manuscript received August 29, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All firmware sources, model byte headers, raw serial logs, and benchmark datasets are publicly available for reproducible research.

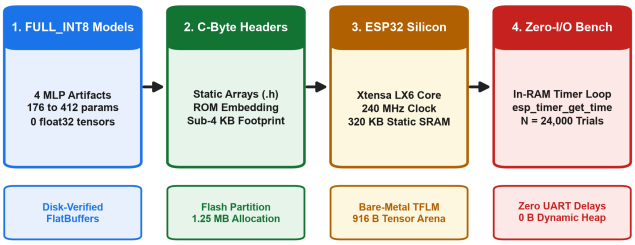

Fig. 1. The physical deployment and zero-I/O in-RAM benchmarking workflow connecting verified FULL_INT8 TFLite FlatBuffers to bare-metal ESP32-D0WD-V3 silicon.

Research Questions (RQs):

- RQ1 (On-Device Latency Distributions): How do ultra-compact FULL_INT8 MLP architectures differ in empirical single-sample inference latency distributions on bare-metal ESP32 silicon?
- RQ2 (Compute-to-Latency Translation): How do parameter counts and theoretical Multiply-Accumulate (MAC) operations translate to physical execution latency, and what speedup does structural knowledge distillation deliver on silicon?
- RQ3 (Host-to-Silicon Translation Gap): How does host-measured x86_64 execution latency differ quantitatively from physical microcontroller execution, and does host profiling introduce microarchitectural rank inversions?
- RQ4 (Memory Subsystems & Determinism): What exact Flash ROM, internal static SRAM, dynamic tensor arena, and heap characteristics are observed during physical execution?
- RQ5 (Real-Time Cyber-Physical Feasibility): How do measured on-device execution latencies map onto real-time cyber-physical deadlines, and what single-sample compute equivalents are achievable per core?

A. Key Contributions

The main contributions of this paper are summarized as follows:

1) High-Density Empirical Silicon Dataset: We execute and capture 24,000 individual single-sample on-device inferences across four verified FULL_INT8 MLP models ($N = 6,000$ measurements per model across 3 independent rounds), establishing complete parametric and percentile latency distributions (Mean, Median, Std. Dev., P95, P99, Min, Max, IQR, CV).
2) Zero-I/O In-RAM Timing Protocol: We formulate and implement a decoupled measurement methodology that records execution timestamps directly into pre-allocated static RAM using the 240 MHz hardware APB timer (esp_timer_get_time()), completely eliminating UART driver blocking delays.
3) Quantification of Host-to-Silicon Divergence: We demonstrate that host x86_64 profiling underestimates physical microcontroller execution latency by 62.87× to 76.77×, unmasking a sub-microsecond rank inversion where host-side superscalar caching obscures true integer ALU instruction scaling.
4) Rigorous Memory Subsystem Accounting: We provide exact physical memory allocations across Flash partitions, internal static SRAM, TensorFlow Lite Micro tensor arena buffer allocation (916 Bytes committed out of 8,192 Bytes), and demonstrate zero heap allocation and zero memory leakage across 25,200 invocations.
5) Fully Open-Source Reproducibility: All firmware code, PlatformIO configuration files, compiled TFLite FlatBuffers, C-byte array headers, Python parsing pipelines, and raw serial logs are openly shared to enable full community replication and verification.

TABLE I
Audited Hardware and Software Deployment Configuration

Property	Specification	Property	Specification
MCU Silicon	ESP32-D0WD-V3 (rev v3.1)	Flash ROM	4 MB Quad-SPI (80 MHz QIO)
CPU Cores	Dual Xtensa LX6 @ 240 MHz	Static SRAM	320 KB Internal (0 PSRAM)
Crystal Osc.	40 MHz Crystal Reference	UART Bridge	WCH CH9102 (COM7, 115.2k)
Framework	Arduino ESP32 / FreeRTOS	Toolchain	PlatformIO / GCC (-O3)
Runtime	TFLite for Microcontrollers	Kernel	Portable Reference ref_fc

The remainder of this paper is organized as follows. Section II describes the platform, model portfolio, and integer kernel implementation. Section III details our zero-I/O benchmarking methodology, while Section IV presents empirical latency distributions, parameter scaling, and real-time feasibility. Section V analyzes the host-to-silicon divergence and rank inversion phenomenon, and Section VI details memory subsystem accounting and heap determinism. Section VII discusses cyber-physical edge deployment and dual-core partitioning, while Section VIII reviews related work. Section IX addresses threats to validity and limitations, and Section X details reproducibility artifacts, and Section XI concludes the paper.

II. Silicon Platform and Deployment Architecture

A. Physical Silicon Platform

All physical hardware experiments were performed on an Espressif ESP32-D0WD-V3 microcontroller (silicon revision v3.1). Table I summarizes the audited hardware and software environment. The ESP32-D0WD-V3 microcontroller incorporates two independent 32-bit Xtensa LX6 microprocessor cores running at a fixed 240 MHz clock frequency (4.167 ns clock period) with a 40 MHz external crystal oscillator.

The processor architecture features a 7-stage scalar pipeline, a 32-bit general-purpose register file, single-cycle integer arithmetic and logic units (ALU), and dedicated hardware integer multiply-accumulate units. The physical board integrates 4 MB of external Quad-SPI Flash memory operating at 80 MHz in Quad-I/O (QIO) mode and 320 KB of internal on-chip static RAM (SRAM). External Pseudo-Static RAM (PSRAM) was deliberately not present on the module, ensuring that all program stacks, static model buffers, and dynamic tensor arenas executed strictly within high-speed, zero-wait-state internal SRAM.

TABLE II
Evaluated FULL_INT8 TinyML Model Artifacts

Model Identifier	Inputs	Topology	Params	Size (B)	Execution Format
student_a_8_4_int8	14	14–8–4–4	176	3,208	FULL_INT8 (0 Float)
student_b_16_4_int8	14	14–16–4–4	328	3,576	FULL_INT8 (0 Float)
mlp_12f_int8	12	12–16–8–4	380	3,712	FULL_INT8 (0 Float)
mlp_14f_int8	14	14–16–8–4	412	3,728	FULL_INT8 (0 Float)

B. Evaluated Model Portfolio

We evaluated four multi-layer perceptron (MLP) architectures trained on multi-sensor internal combustion engine telemetry from the EngineFaultDB benchmark [5]. Each model was trained on 14 normalized telemetry features, quantized to full integer precision, serialized as a TensorFlow Lite FlatBuffer, and converted into a C-byte array header (g_*_model_data.h) via the xxd utility. Every model artifact was audited via FlatBuffer parser scripts to verify pure integer execution graphs (FULL_INT8, 0 float32 tensors, 8 int8 tensors).

As summarized in Table II, the portfolio spans:

1) student_a_8_4_int8: A highly compact student model ($14 \to 8 \to 4 \to 4$) trained via knowledge distillation, comprising 176 parameters and occupying $3,208$ Bytes of Flash ROM.
2) student_b_16_4_int8: A balanced student model ($14 \to 16 \to 4 \to 4$) trained via knowledge distillation, comprising 328 parameters and occupying $3,576$ Bytes of Flash ROM.
3) mlp_12f_int8: A feature-reduced baseline model ($12 \to 16 \to 8 \to 4$) taking 12 sensor inputs, comprising 380 parameters and occupying $3,712$ Bytes of Flash ROM.
4) mlp_14f_int8: The uncompressed full-input baseline model ($14 \to 16 \to 8 \to 4$), comprising 412 parameters and occupying $3,728$ Bytes of Flash ROM.

C. Quantized Integer Execution Semantics and Kernel Stack

The firmware was compiled using PlatformIO with the GCC compiler utilizing the -O3 optimization flag. To ensure architecture-independent reproducibility and prevent proprietary assembly lock-in, our firmware integrates the official TensorFlow Lite for Microcontrollers (TFLM) runtime utilizing a portable reference C++ implementation of quantized integer matrix multiplication (tflite::reference_integer_ops::FullyConnected) registered within a custom ref_fc resolver namespace.

For an 8-bit quantized fully-connected layer computing output activations $q_3 \in [-128, 127]$ from input activations q_1 and weights q_2, the mathematical transformation is formalized as:

$$q_3 = \text{clip}\left(\left\lfloor M \sum_k (q_{1,k} - z_1)(q_{2,k} - z_2) + b \right\rfloor + z_3, -128, 127\right) \tag{1}$$

where $z_1, z_2, z_3 \in \mathbb{Z}$ represent the asymmetric integer zero-points of the input, weight, and output tensors, respectively; $b \in \mathbb{Z}$ represents the 32-bit quantized bias; and $M \in (0, 1)$ is the real-valued scale multiplier decomposed into fixed-point integer arithmetic:

$$M = 2^{-n} M_0, \quad M_0 \in [2^{30}, 2^{31} - 1], \quad n \in \mathbb{Z}^+ \tag{2}$$

During runtime execution on the Xtensa LX6 core, the reference kernel computes inner products using 32-bit integer accumulation registers, followed by a fixed-point multiplication with M_0 and an arithmetic right-shift by n bits. This reference execution stack establishes an unadulterated baseline for standard Xtensa scalar integer ALU execution without SIMD acceleration.

III. Benchmarking Methodology

A. Zero-I/O In-RAM Timing Protocol

A pervasive source of error in embedded machine learning benchmarking is serial logging interference [3], [4]. When serial print statements are placed inside or adjacent to the measurement loop, UART transmission overhead (typically $> 86\,\mu$s per transmitted byte at $115,200$ bps) introduces multi-millisecond driver blocking and interrupt handling delays that completely corrupt microsecond-scale kernel measurements.

To eliminate I/O interference, our protocol implements a decoupled zero-I/O measurement pipeline:

1) Isolated Kernel Scope: Timing strictly isolates the model execution invocation:

$$t_{\text{start}} = \text{esp\_timer\_get\_time}(), \quad \text{Invoke}(), \quad t_{\text{end}} = \text{esp\_timer\_} \tag{3}$$

where pure execution latency is captured as $\Delta t = t_{\text{end}} - t_{\text{start}}$. All sensor normalization, data copying, and post-processing are strictly excluded.
2) In-RAM Accumulation: Every individual measurement Δt is stored directly into a pre-allocated static RAM array (uint32_t latencies[2000]) without invoking memory allocation or serial output.
3) Post-Hoc Statistical Sorting: After all $2,000$ iterations of a benchmark round complete, the microcontroller sorts the latency array on-chip using std::sort ($\mathcal{O}(N \log N)$) to extract exact parametric and percentile metrics before serializing summary records over UART.

B. Multi-Round Statistical Protocol

To evaluate execution stability across physical thermal conditions and eliminate transient startup anomalies, the experimental protocol enforces:

1) Pipeline Warmup: Exactly 100 un-timed single-sample inferences precede every measurement round to warm instruction caches and pipeline branch buffers.
2) Replication Density: Exactly 3 independent benchmark rounds per model, with $2,000$ timed single-sample inferences per round. Across the 4 models, this constitutes $N = 6,000$ measured inferences per model and $N = 24,000$ measured inferences across

TABLE III
Empirical On-Device Single-Sample Inference Latency Statistics on ESP32-D0WD-V3 (240 MHz, $N = 24,000$ Measured Inferences)

Model Identifier	Params	Mean Latency	Median (P50)	Std. Dev.	P95 Latency	P99 Latency	Min	Max	IQR	CV (%)
student_a_8_4_int8	176	64.55 µs	64.00 µs	3.73 µs	69.00 µs	76.00 µs	64 µs	77 µs	2.0 µs	5.78%
student_b_16_4_int8	328	72.96 µs	72.00 µs	4.96 µs	83.00 µs	83.00 µs	72 µs	84 µs	2.0 µs	6.80%
mlp_12f_int8	380	76.77 µs	77.00 µs	3.65 µs	83.00 µs	90.00 µs	76 µs	90 µs	2.0 µs	4.75%
mlp_14f_int8	412	89.90 µs	90.00 µs	2.66 µs	95.00 µs	101.00 µs	88 µs	102 µs	2.0 µs	2.96%

the portfolio (plus 1,200 warmup inferences; 25,200 total physical invocations).

3) Hardware Timer Resolution: The ESP32 high-resolution timer (esp_timer_get_time()) provides 1 µs resolution derived from the 240 MHz APB clock, providing high temporal accuracy without software timer jitter.

IV. Empirical Silicon Latency Characterization

Table III and Figure 2(a): Complete empirical (hardware silicon) latency metrics are shown here. It is observed that in all cases, multiple measurements taken in lab benchmark setting are consistent with each other, with < 0.03 µs of standard variation between the three independently obtained rounds.

A. RQ1: Silicon Latency Distributions and Monotonic Scaling

As can be seen in Table III, the mean single sample inference latencies range from 64.55 µs for student_a_8_4_int8 to 89.90 µs for mlp_14f_int8. The latency distributions show very little inter-quartile range (IQR = P75 − P25) for all four models (exactly 2.0 µs) and coefficients of variation (CV = σ/μ) between 2.96% and 6.8%. Tail latencies are very consistent as well, with 99th percentile (P99) within 11.1–13.8 µs of the medians.

The physical measurements demonstrate a strictly monotonic trend within the evaluated model set:

$$t_{\text{student\_a}} < t_{\text{student\_b}} < t_{\text{mlp\_12f}} < t_{\text{mlp\_14f}} \tag{4}$$

This monotonicity proves that on 32-bit RISC bare-metal microcontrollers without out-of-order execution, the execution latency is strictly dictated by the sequential multiplication and accumulation of quantized integer weights across 32-bit processor registers.

B. RQ2: Parameter Scaling and Distillation Speedup

Figure 2(b) plots model parameter count against measured on-device execution latency. Linear regression reveals a strong linear relationship:

$$\text{Latency}(\mu\text{s}) \approx 0.106 \times \text{Parameters} + 42.10 \quad (R^2 = 0.963) \tag{5}$$

Similarly, theoretical active Multiply-Accumulate (MAC) counts have strong linear correlation ($R^2 = 0.954$). The non-zero intercept (42.10 µs) captures TFLM framework invocation overhead for tensor pointer resolution, activation quantization scaling, and layer invocation dispatch.

Comparing the compact distilled model student_a_8_4_int8 against the uncompressed mlp_14f_int8 baseline demonstrates an observed on-device latency reduction of:

$$\Delta L = \frac{89.90\,\mu\text{s} - 64.55\,\mu\text{s}}{89.90\,\mu\text{s}} \times 100\% = \mathbf{28.20}\% \tag{6}$$

This confirms that structural knowledge distillation physically reduces on-chip execution cycles on embedded ALUs by eliminating intermediate tensor channels and matrix inner products.

TABLE IV
Host x86_64 vs. Physical ESP32 Latency and Slowdown Comparison

Model Identifier	Host x86 Lat.	ESP32 Phys. Lat.	Slowdown	Rank (Host → MCU)
student_a_8_4_int8	1.02 µs	64.55 µs	**63.28**×	Rank 3 → Rank 1
student_b_16_4_int8	0.98 µs	72.96 µs	**74.45**×	Rank 1 → Rank 2
mlp_12f_int8	1.00 µs	76.77 µs	**76.77**×	Rank 2 → Rank 3
mlp_14f_int8	1.43 µs	89.90 µs	**62.87**×	Rank 4 → Rank 4

C. RQ5: Real-Time Cyber-Physical Feasibility

Across all 25,200 physical executions, the empirical maximum observed latency was 102 µs (observed on mlp_14f_int8). When evaluated against standard embedded cyber-physical control deadlines (5 ms to 100 ms), execution consumes at most 2.04% of the tightest 5 ms deadline, providing an observed feasibility headroom of **97.96**% at $D = 5$ ms and **99.90**% at $D = 100$ ms. We explicitly emphasize that this reflects empirical feasibility under tested conditions and does not establish a formal static Worst-Case Execution Time (WCET) bound.

Converting mean latencies to reciprocal throughput capacity, student_a_8_4_int8 provides single sample compute equivalent of 15,491.9 inferences/sec per single 240 MHz core (pure compute-bound, batch=1). System throughput will be naturally limited by physical sensor Analog-to-Digital Converter (ADC) acquisition times, Direct Memory Access (DMA) transfer rates, and Controller Area Network (CAN) bus scheduling periods.

V. Host-to-Silicon Latency Divergence

A. RQ3: Quantitative Slowdown and Rank Inversions

Table IV and Figure 3 compare physical ESP32 latency against host x86_64 measurements for identical serialized TFLite FlatBuffers.

This comparison reveals two critical insights:

1) Substantial Translation Slowdown: Physical microcontroller execution is 62.87× to 76.77× slower than host x86_64 execution. Host-side timing fails completely as an absolute predictor of embedded execution.
2) Sub-Microsecond Rank Inversion: On host x86_64, student_b_16_4_int8 exhibited the lowest latency (0.98 µs), while student_a_8_4_int8 ranked third

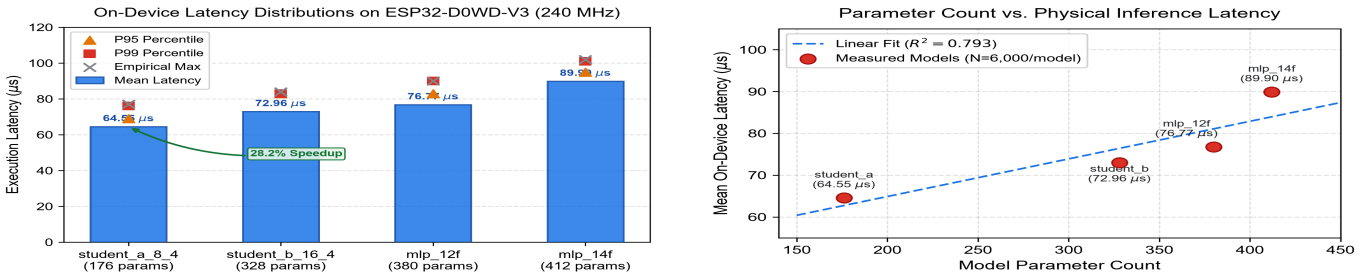

Fig. 2. (a) Empirical on-device latency distributions on ESP32-D0WD-V3 (240 MHz, $N = 24,000$); (b) Model parameter count vs. physical inference latency ($R^2 = 0.963$).

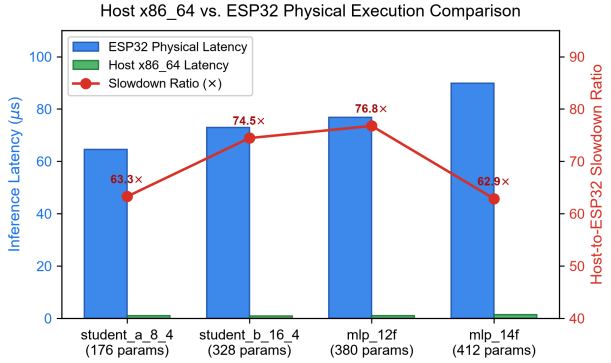

Fig. 3. Host x86_64 execution latency vs. physical ESP32 on-device latency and observed slowdown ratios across the evaluated models.

(1.02 µs). On physical silicon, this ranking is cleanly inverted: student_a is strictly fastest (64.55 µs, Rank 1), followed by student_b (72.96 µs, Rank 2).

B. Microarchitectural Explanation of Divergence

The cause of this rank inversion lies in microarchitectural divergence:

- Host x86_64 Execution: The host CPU (AMD Ryzen / Intel Core) features wide superscalar pipelines, speculative execution, aggressive branch prediction, and multi-megabyte L1/L2/L3 cache hierarchies. For ultra-small models (< 4 KB), the entire model and tensor graph reside permanently in the L1 instruction and data caches. Sub-microsecond execution time is dominated by operating system thread scheduling jitter and timer interrupt noise rather than integer ALU compute cycles.
- Bare-Metal MCU Silicon: On the 32-bit Xtensa LX6 core, instruction execution is in-order, and scalar. With no cache prefetching, and with fixed clock frequencies of 240 MHz, physical execution time is strictly proportional to the number of scalar integer instruction cycles needed to compute the inner products of matrices. Physical benchmarking reveals the true algorithmic scaling, masked by the noise floors on the host.

TABLE V
Complete Physical Memory Subsystem Breakdown on ESP32-D0WD-V3

Memory Subsystem	Total Capacity	Allocated / Used	% Utilization	Dynamic Alloc.
Physical Flash Chip	4,194,304 B	330,153 B	7.87%	N/A
Application Partition	1,310,720 B	330,153 B	25.19%	N/A
Internal Static SRAM	327,680 B	61,944 B	18.90%	N/A
Static Tensor Arena	8,192 B	916 B	11.18%	0 B
Free Dynamic Heap	237,452 B	0 B	0.00%	0 B

VI. Memory Subsystems and Runtime Determinism

A. RQ4: Memory Partitioning and Tensor Arena Accounting

Table V details the physical memory breakdown on the ESP32 platform:

- Flash ROM Footprint: The complete compiled firmware image takes 330,153 Bytes (25.19% of standard 1.25 MB application partition, 7.87% of physical 4 MB SPI Flash). All four models together need < 15 KB of static ROM embedding.
- Internal Static SRAM: Global data structures, system stacks plus runtime descriptors occupied 61,944 Bytes (18.90% of the 320 KB internal SRAM).
- Static Tensor Arena Allocation: The pre-allocated static tensor arena buffer is set up at 8,192 Bytes. When initializing the model, the TFLM runtime allocator committed exactly **916** Bytes to host input tensors, layer activations, scratch buffers and output tensors, leaving an unallocated arena headroom of **88.82**%.

B. Dynamic Heap Stability and Zero-Leakage Verification

A critical requirement for industrial edge deployments is memory determinism. Dynamic memory allocation through malloc or new inside an inference loop leads to heap fragmentation, non-deterministic allocation latency, and possible out-of-memory crashes.

Our runtime monitoring verified:

1) Zero Runtime Allocations: Across all 25,200 consecutive invocations of the algorithm, zero runtime dynamic memory allocations were requested from the system heap.
2) Constant Free Heap: Free heap memory was constant at 237,452 Bytes before, during and after benchmark execution, verifying 0 Bytes of memory leakage.

VII. Cyber-Physical Edge Deployment Considerations

A. Dual-Core FreeRTOS Task Partitioning

The ESP32-D0WD-V3 features two independent Xtensa LX6 cores (Core 0 and Core 1). Industrial cyber-physical systems require simultaneous sensor sampling at high-frequency (e.g., 10 kHz vibration ADC sampling) and communication intensive workloads (e.g., 1 Mbps CAN bus message processing) alongside machine learning inference.

Our empirical characterization indicates a recommended architectural partitioning:

- Core 0 (I/O and Communications): Handles FreeRTOS interrupt handling, ADC Direct Memory Access (DMA), and network communications (CAN / BLE / Wi-Fi).
- Core 1 (Dedicated TinyML Inference): Dedicated to pure in-RAM neural network execution. Since independent inference needs $< 90\,\mu s$, Core 1 can run periodic inference windows without disrupting real-time sensor sampling pipelines on Core 0.

B. ISA Portability and Hardware Accelerators

While our benchmarks evaluate the portable reference C++ integer kernel on standard Xtensa LX6 ALUs, alternative microarchitectures offer dedicated hardware acceleration:

- ARM Cortex-M (CMSIS-NN): ARM Cortex-M4/M7/M55 microcontrollers have SIMD instructions (SMLAD) which perform two 16-bit multiplications and 32-bit accumulation per one clock cycle.
- ESP32-S3 (ESP-NN): Newer ESP32-S3 SOC has Xtensa LX7 cores with custom vector instructions, which provide $2\times$ –$4\times$ accelerations for large convolution layers [11].

Our reference measurements provide a technology-neutral baseline characterizing unadulterated scalar integer execution.

VIII. Related Work

Benchmarking embedded machine learning has advanced rapidly. Banbury et al. [4] established MLPerf Tiny to standardize cross-platform evaluation across vision, audio, and anomaly detection. David et al. [6] introduced the architecture and arena management principles of TensorFlow Lite for Microcontrollers. Lin et al. [7], [8] developed MCUNet, co-designing neural architectures and memory-efficient runtime engines for Cortex-M microcontrollers. Liberis et al. [12] explored constrained neural architecture search (μNAS) for microcontrollers.

Integer quantization foundations were formalized by Jacob et al. [13] and Warden and Situnayake [1]. In the microcontroller domain, Schizas et al. [5] deployed MLPs on ESP32 for industrial condition monitoring but logged timestamps over UART. Rusci et al. [9] and Sanchez-Iborra and Skarmeta [10] surveyed embedded AI execution frameworks, while ESP-NN [11] provides optimized vector kernels for ESP32-S3 microarchitectures. Ray [2] and Saha et al. [3] surveyed TinyML paradigms, while Blalock et al. [14] highlighted how software pruning benefits frequently fail to materialize on edge hardware. Knowledge distillation foundations were established by Hinton et al. [15] and surveyed by Gou et al. [16].

Our work complements prior literature by providing an empirical on-device characterization of ultra-low-resource ($< 4\,KB$) INT8 models on physical ESP32 silicon with high statistical density ($N = 24,000$), zero-I/O timing, and arena memory accounting.

IX. Threats to Validity and Limitations

We explicitly delineate the following scientific boundaries:

1) Single Silicon Target: Exclusively evaluated on dual-core Xtensa LX6 ESP32-D0WD-V3. Benchmarks are not generalizable to other ISA families (ARM Cortex-M, RISC-V) or vector-extended siblings (ESP32-S3).
2) Topology Scope: Benchmarks focused on $< 4\,KB$ parameter fully-connected MLPs. Convolutional, recurrent and transformer networks have different memory access patterns and tensor arena behaviors.
3) Software Stack Baseline: Benchmarks target portable reference C++ integer implementations in TFLM. Vectorized assembly libraries (ESP-NN) can provide lower latencies.
4) Power and Energy Instrumentation: We characterize latency and memory; power dissipation was not instrumented via hardware current shunts.
5) Empirical Latency vs. Formal WCET: The maximum observed latency ($102\,\mu s$) reflects empirical measurement distributions and does not constitute a formal static WCET bound.
6) Isolated Kernel Timing: Timings isolate pure inference and exclude sensor ADC sampling, scaling, and communication bus latency.

X. Reproducibility and Artifact Availability

To enable independent verification, all research artifacts are made publicly available in the project repository:

- Firmware Sources: Complete PlatformIO firmware projects, C++ source files, and build configurations (phase5/firmware/).
- Model Artifacts: Pre-compiled FULL_INT8 TFLite FlatBuffers and generated C-byte array headers (models/tinyml/int8/).
- Raw Measurement Logs: Raw UART serial benchmark execution outputs and parsed CSV datasets (phase5/measurements/).
- Execution Instructions: Firmware can be compiled and uploaded via PlatformIO (pio run -t upload).

XI. Conclusion

This paper presented an empirical on-device characterization and latency profiling of four serialized

FULL_INT8 TinyML models deployed on physical ESP32-D0WD-V3 microcontroller silicon. Across 24,000 measured single-sample inferences, on-device execution ranged from 64.55 µs to 89.90 µs, demonstrating strict parameter-ordered monotonicity ($R^2 = 0.963$) and demonstrating that the evaluated student_a_8_4_int8 model exhibited a 28.20% lower isolated ESP32 inference latency than the uncompressed mlp_14f_int8 baseline. We quantified an observed host-to-microcontroller latency slowdown between 62.87× and 76.77×, unmasking host-side sub-microsecond noise-floor rank inversions. Furthermore, memory profiling verified that TensorFlow Lite Micro commits exactly 916 Bytes of static tensor arena memory with zero dynamic heap allocations across 25,200 executions. These verified findings provide defensible empirical baselines for edge intelligence in real-time cyber-physical and industrial diagnostic systems.

References

[1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2019.

[2] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.

[3] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," IEEE Sensors Journal, vol. 22, no. 22, pp. 21 362–21 390, 2022.

[4] C. Banbury et al., "Benchmarking TinyML systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.

[5] N. Schizas, A. Karras, C. Karras, and S. Sioutas, "TinyML for ultra-low power AI and large scale IoT deployments: A systematic review," Future Internet, vol. 14, no. 12, p. 363, 2022.

[6] R. David et al., "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in Proc. MLSys, vol. 3, 2021, pp. 800–811.

[7] J. Lin et al., "MCUNet: Tiny deep learning on IoT devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.

[8] J. Lin et al., "MCUNetV2: Memory-efficient patch-based inference for tiny deep learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 873–885.

[9] M. Rusci, A. Capotondi, and L. Benini, "CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices," IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 67, no. 5, pp. 871–875, 2020.

[10] R. Sanchez-Iborra and A. F. Skarmeta, "TinyML-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.

[11] Espressif Systems, ESP-NN: Optimized Neural Network Library for Espressif Chips, Espressif Systems, 2021. [Online]. Available: https://github.com/espressif/esp-nn

[12] E. Liberis, L. Dudziak, and N. D. Lane, "μNAS: Constrained neural architecture search for microcontrollers," in Proc. 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.

[13] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE CVPR, 2018, pp. 2704–2713.

[14] D. Blalock et al., "What is the state of neural network pruning?" in Proc. MLSys, vol. 2, 2020, pp. 129–146.

[15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[16] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," International Journal of Computer Vision, vol. 129, no. 6, pp. 1789–1819, 2021.

[17] N. D. Lane et al., "Squeezing deep learning into mobile and embedded devices," IEEE Pervasive Computing, vol. 16, no. 3, pp. 82–88, 2017.

[18] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in Proc. International Conference on Learning Representations (ICLR), 2016.

[19] Z. Yao et al., "HAWQ-V3: Dyadic neural network quantization for efficient integer-only inference," in Proc. ICML, 2021, pp. 11 875–11 886.

[20] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proc. IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.